

MINISTRY OF EDUCATION OF REPUBLIC OF MOLDOVA
TECHNICAL UNIVERSITY OF MOLDOVA
FACULTY OF COMPUTERS, INFORMATICS AND MICROELECTRONICS
SOFTWARE ENGINEERING DEPARTMENT

EMBEDDED SYSTEMS
LABORATORY WORK #7 - PART 1

Actuator Control System on Arduino Uno

Author:

Sava LUCHIAN
std. gr. FAF-233

Verified:

Alexei MARTINIUC

Chisinau 2026

Technical Task

Purpose of the Laboratory

The purpose of the laboratory is to study actuator control and command-signal conditioning on an embedded microcontroller by implementing a modular application on **Arduino Uno**. The system receives user commands through STDIO over the serial terminal, controls one binary actuator and one analog actuator, displays current status on a 16×2 LCD, and reports structured runtime data using `printf()`.

Objectives

1. Implement a binary actuator channel that accepts ON/OFF commands through STDIO and validates state changes through software debouncing.
2. Implement an analog actuator channel that accepts percentage commands and applies saturation, low-pass filtering, and soft ramping before driving a stepper motor.
3. Use a 16×2 LCD to present current actuator state and active alerts.
4. Use `printf()` redirected to UART as the main reporting interface.
5. Structure the application in dedicated modules: configuration, I/O, signal conditioning, actuators, and application logic.
6. Validate the implementation in Wokwi and build it with PlatformIO for Arduino Uno.

Individual Task

Design and implement on an **Arduino Uno (ATmega328P)** a bare-metal actuator-control application for **Part 1** of the assignment, covering the same integrated binary and analog actuator demonstrator used during development:

- **Binary actuator** – digital indicator channel on D12, controlled from serial commands `bin on`, `bin off`, and `bin toggle`.
- **Analog actuator** – stepper-motor speed channel driven through a software phase driver on D8–D11, controlled from serial commands `ana <0..100>`.
- **Signal conditioning** – debounce for the binary command path; saturation, low-pass filtering, and ramp limiting for the analog path.

- **LCD interface** – displays binary state, pending transition, stepper speed percentage, RPM estimate, position, and alert summary.
- **STDIO interface** – all text output is performed with `printf()` redirected to UART through `fdev_setup_stream`.
- **Timing** – 20 ms control period and 500 ms reporting period, satisfying the required sub-100 ms response time.

1 Domain Analysis

1.1 Technological Context and Application Domain

Actuators transform electrical control signals into physical actions. In simple embedded systems, the most common actuator-control categories are binary switching and variable motion control. Binary actuators are suitable for simple enable/disable behaviour, while stepper-based actuator control is required when the application must regulate rotational speed and position progression through timed phase sequencing.

This laboratory combines both categories in one application. The binary channel demonstrates command debouncing and stable state validation, while the analog channel demonstrates command saturation, filtering, and ramping. The result is a compact but representative example of command generation, signal conditioning, and actuator interfacing.

1.2 Hardware Components and Justification

Component	Qty	Role in the application
Arduino Uno (ATmega328P)	1	Main MCU running the cooperative control loop, serial I/O, LCD output, and actuator drivers.
16×2 LCD (HD44780 compatible)	1	Local user interface for current binary state, stepper speed, position, and alerts.
Yellow LED + 220 Ω	1	Binary actuator indicator on D12.
28BYJ-48 5V stepper motor + ULN2003 driver board	1	Analog actuator channel driven from D8–D11 with a software half-step sequence.
USB serial terminal	1	User command entry and <code>printf()</code> reporting channel.
Breadboard / Wokwi wiring	–	Physical or simulated interconnection medium.

Table 1: Hardware components used in Lab 7

1.3 Software Components and Technologies

- **PlatformIO + VS Code:** single uno environment, Arduino framework.
- **Arduino framework:** GPIO, timing functions, and serial communication.
- **LiquidCrystal 1.0.7:** LCD driver library.
- **AVR-libc STDIO:** `stdout` redirection through `fdev_setup_stream` so that runtime output uses `printf()`.
- **Wokwi:** simulation of Arduino Uno, LCD, and LED-based actuator demonstrators.
- **PlantUML:** source format used to define diagrams that will later be rendered to PNG files.

1.4 Analysis of Existing Solutions

Several implementation options were considered:

- **Bare-metal cooperative loop vs. RTOS:** the application contains only two periodic activities and a serial command parser, so a `millis()`-based loop is simpler and more memory-efficient than FreeRTOS on ATmega328P.

- **Direct command application vs. conditioned command path:** the final design conditions both command paths before actuating the outputs, which matches the theoretical requirements better than immediate state changes.
- **Real motor-driver setup vs. simplified indicators:** the final design uses a simple LED as the binary actuator and a 28BYJ-48 stepper path driven through a ULN2003-labeled control section in the simulation.
- **Serial-only vs. mixed interfaces:** the final version retains terminal control and `printf()` reporting as the primary interface, which satisfies the STDIO requirement directly.

1.5 Real-World Applications

The same software pattern appears in many practical systems:

- **Lighting control:** binary enable/disable plus brightness control.
- **Positioning and dispensing systems:** binary enable indication plus stepper-based transport or positioning control.
- **Pump systems:** binary state indication plus motor-speed command generation.
- **Laboratory demonstrators:** command validation and actuator-state visualisation on low-cost MCU platforms.

2 Conceptualisation and Design

2.1 Technical Requirements

ID	Category	Requirement	Type
R-F-01	STDIO	System shall accept user commands through serial terminal input.	Func.
R-F-02	Binary actuator	System shall support <code>bin on</code> , <code>bin off</code> , and <code>bin toggle</code> .	Func.
R-F-03	Binary actuator	Binary commands shall pass through software debounce before state commitment.	Func.
R-F-04	Analog actuator	System shall support <code>ana <0..100></code> percentage commands.	Func.
R-F-05	Analog conditioning	Analog command shall be saturated to the interval 0..100%.	Func.
R-F-06	Analog conditioning	Analog command shall be filtered by a digital low-pass stage.	Func.
R-F-07	Analog conditioning	Analog output shall be ramp-limited for smooth transitions.	Func.
R-F-08	Stepper drive	Conditioned analog output shall be converted to a timed stepper-drive sequence on D8–D11.	Func.

Table 2: Technical requirements for Lab 7 – Part 1

ID	Category	Requirement	Type
R-F-09	LCD	LCD shall show binary state, pending state, stepper output percent, RPM estimate, and position.	Func.
R-F-10	LCD	LCD shall show saturation/high-output alert status and reporting state.	Func.
R-F-11	STDIO	System shall print startup help and runtime status via <code>printf()</code> .	Func.
R-F-12	STDIO	System shall support status , report on , and report off .	Func.
R-NF-01	Timing	Control update period shall be 20 ms or faster.	Non-func.
R-NF-02	Latency	User-visible response shall remain below 100 ms.	Non-func.
R-NF-03	Modularity	Software shall be split into dedicated modules for app, I/O, actuators, signal, and config.	Non-func.
R-NF-04	Platform	Firmware shall fit Arduino Uno memory limits.	Non-func.
R-NF-05	Tooling	Project shall be buildable with PlatformIO and simulatable in Wokwi.	Non-func.

Table 3: Technical requirements for Lab 7 – Part 2

2.2 Architectural Design

2.2.1 System Structural Architecture

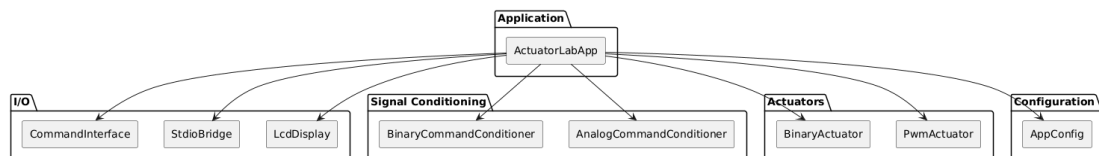


Figure 1: Layered software architecture for Lab 7

2.2.2 HW/SW Interface and Component Interaction

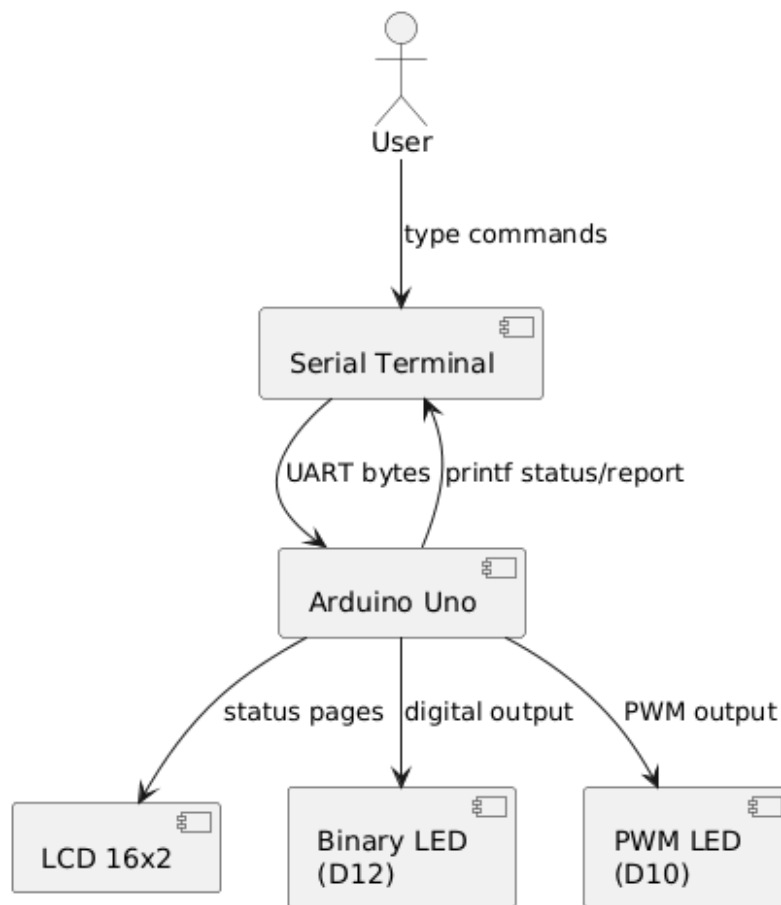


Figure 2: Component interaction diagram for Lab 7

2.2.3 Actuator Control Pipeline

Stage	Purpose
Serial command input	Receives textual commands from terminal through UART.
Command parsing	Converts textual command into a binary request or analog target value.
Binary conditioning	Adds debounce and stable-state validation for ON/OFF requests.
Analog conditioning	Applies saturation, low-pass filtering, and ramp limiting.
Actuator driver layer	Maps binary state to a digital output and analog percent to stepper stepping frequency.
LCD/STDIO	Exposes internal state to the user in human-readable format.

Table 4: Actuator control pipeline used in Lab 7

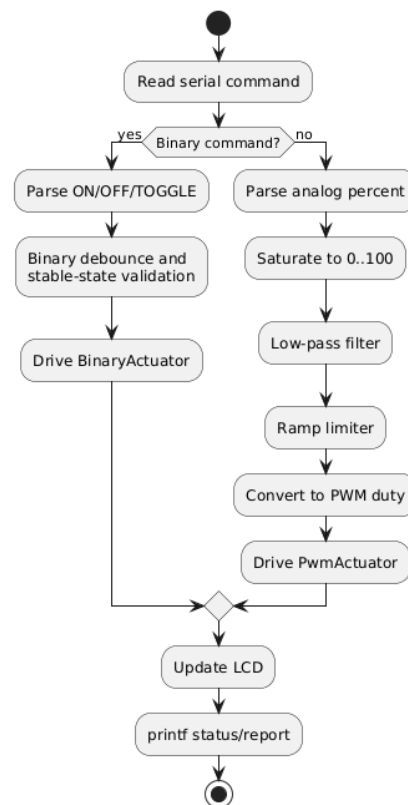


Figure 3: Binary and analog actuator control pipeline

2.2.4 Main Loop Timing Design

Activity	Period	Trigger	Handler
Command polling	every loop	serial bytes available	<code>processCommands()</code>
Control update	20 ms	<code>millis()</code> elapsed	<code>runControlTask()</code>
LCD page update	2000 ms	<code>millis()</code> elapsed	<code>updateLcd()</code>
Status reporting	500 ms	<code>millis()</code> elapsed	<code>runReportTask()</code>

Table 5: Multi-rate timing model

2.2.5 STDIO Bridge Design

On AVR, `printf()` writes to `stdout`. Because `stdout` is not connected to UART by default, the project uses `StdioBridge::begin()` to install a custom `putchar` callback using `fdev_setup_stream`. Each output character is forwarded to `Serial.write()`, enabling formatted serial reporting entirely through `printf()`.

2.3 Behavioural Modelling

2.3.1 Application Tick Flowchart

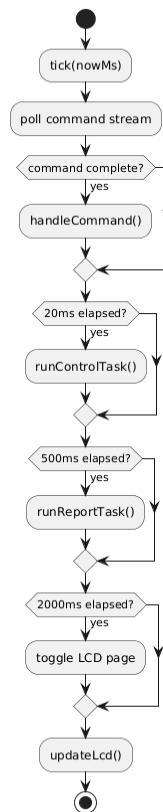


Figure 4: Flowchart of the application tick loop

2.3.2 Command Processing Flowchart

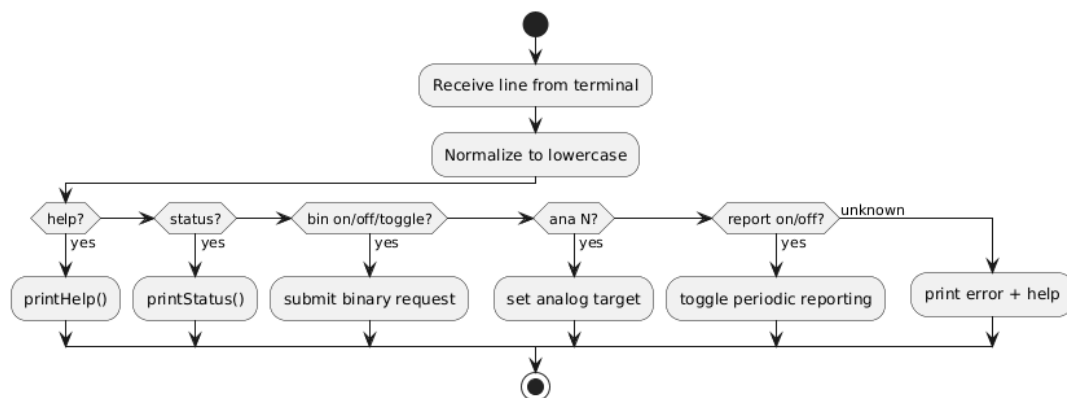


Figure 5: Flowchart of serial command decoding and dispatch

2.3.3 Sequence Diagram – One Command Cycle

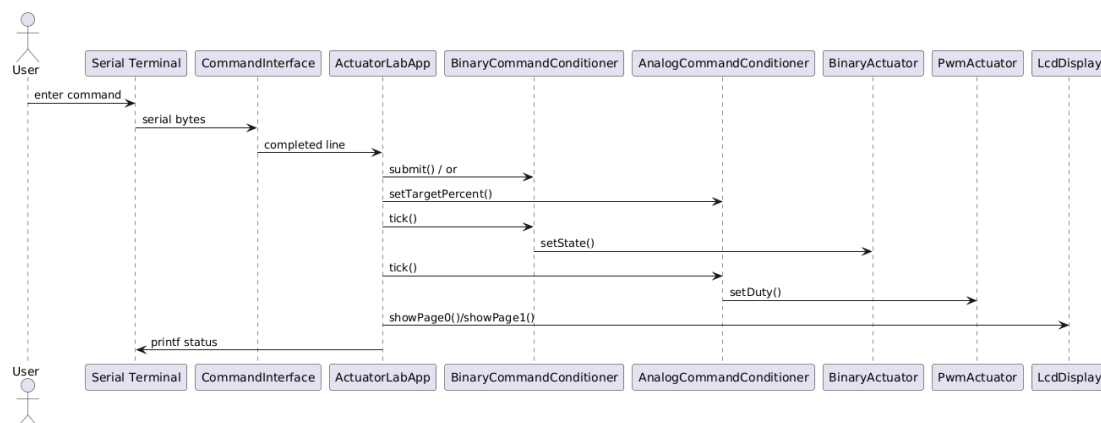


Figure 6: Sequence diagram for one command-processing cycle

2.3.4 System State Machine

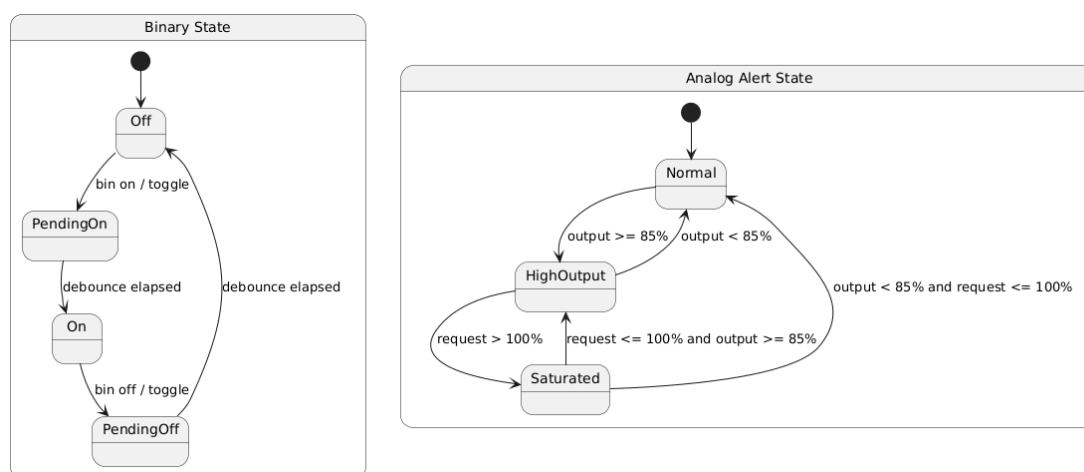


Figure 7: System state machine with binary stable/pending states and analog alert flags

2.4 Test Scenarios and Validation Criteria

ID	Category	Scenario	Expected result	Req.
T-01	Startup	Reset the board	Startup banner and help text printed once	R-F-11
T-02	Binary ON	Send bin on	Yellow LED turns ON after debounce delay	R-F-02, R-F-03
T-03	Binary OFF	Send bin off	Yellow LED turns OFF after debounce delay	R-F-02, R-F-03
T-04	Binary toggle	Send bin toggle twice	Binary state toggles correctly on each accepted command	R-F-02
T-05	Analog nominal	Send ana 30	Stepper speed ramps smoothly to a low level	R-F-04, R-F-06, R-F-07
T-06	Analog high	Send ana 85	Stepper reaches a high rotation rate and high alert becomes active	R-F-04, R-F-10
T-07	Analog saturation	Send ana 120	Command is clamped to 100% and saturation alert is reported	R-F-05, R-F-10

Table 6: Test scenarios and validation criteria – Part 1

ID	Category	Scenario	Expected result	Req.
T-08	Manual status	Send status	One structured STAT ... line is printed	R-F-11, R-F-12
T-09	Periodic reporting ON	Send report on	Status lines appear every 500 ms	R-F-12
T-10	Periodic reporting OFF	Send report off	Periodic lines stop, manual commands still work	R-F-12
T-11	LCD page 0	Observe normal display page	LCD shows binary state, pending mark, stepper percent, RPM, and position	R-F-09
T-12	LCD page 1	Wait for page rotation	LCD shows saturation/high alert and report state	R-F-10
T-13	Enter handling	Press Enter in VS Code Wokwi terminal	Both CR and LF execute commands correctly	R-F-01
T-14	Backspace handling	Type and edit a command before Enter	Command line editing works in terminal	R-F-01
T-15	Build resources	Build with PlatformIO	Flash and RAM remain within Uno limits	R-NF-04

Table 7: Test scenarios and validation criteria – Part 2

2.5 Design Phase Conclusions

The final design is intentionally small and modular. The binary and analog channels share the same application shell, but each has its own conditioning logic and output driver. This makes the project suitable both as a laboratory demonstrator and as a base for replacing the binary indicator with another digital load and keeping the ULN2003 stepper drive path for physical hardware later.

3 Hardware and Software Implementation

3.1 Electrical Schematic

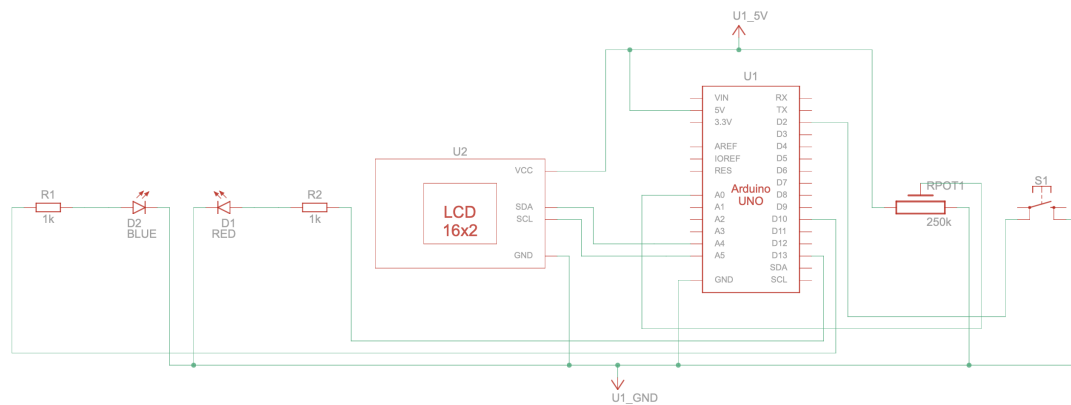


Figure 8: Electrical schematic / Wokwi wiring used in Lab 7

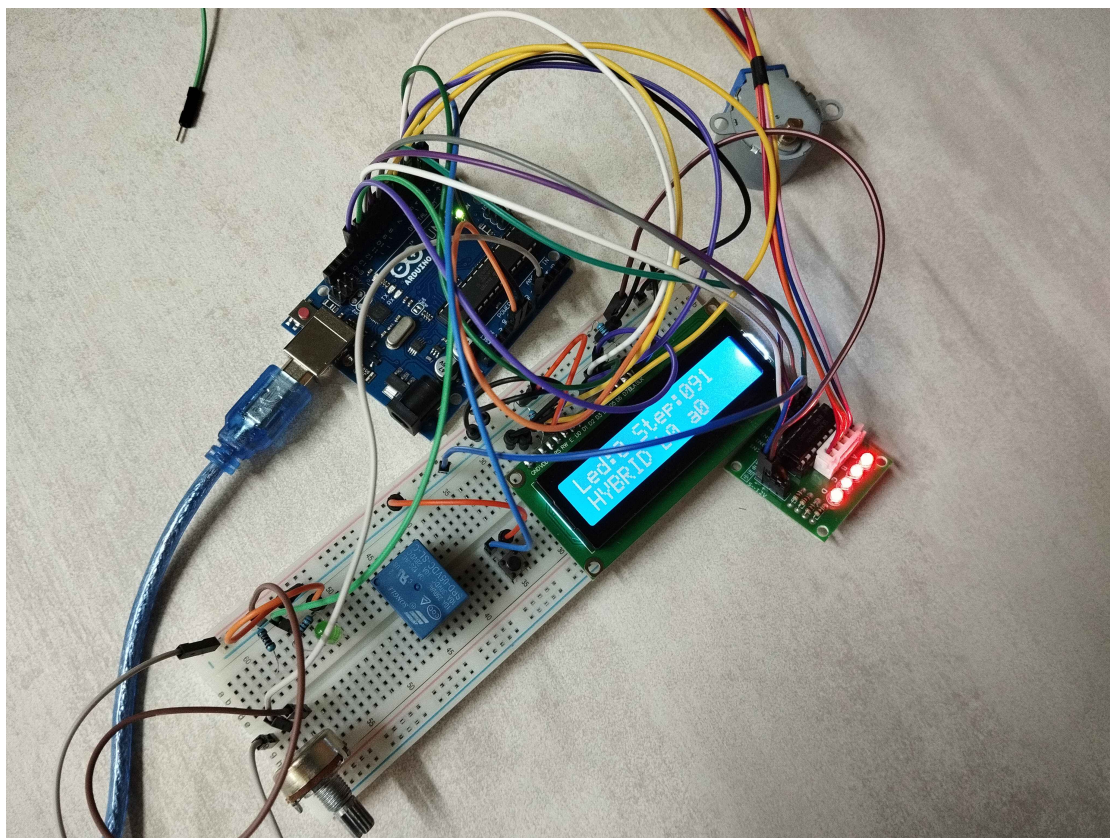


Figure 9: Physical hardware setup used for Lab 7, Part 1

Pin assignment:

- D7 – LCD RS
- D6 – LCD Enable

- D5 – LCD D4
- D4 – LCD D5
- D3 – LCD D6
- D13 – LCD D7
- D8–D11 – stepper phase outputs
- D12 – binary actuator digital output
- D0/D1 – USB serial UART for `STDIO` input and `printf()` output

3.2 Project Directory Structure

```
lab7/
|-- platformio.ini
|-- diagram.json
|-- wokwi.toml
|-- src/
|   |-- main.cpp
|   |-- config/
|   |   '-- AppConfig.h
|   |-- actuators/
|   |   |-- BinaryActuator.h/.cpp
|   |   '-- Stepper28BYJ48Actuator.h/.cpp
|   |-- signal/
|   |   |-- BinaryCommandConditioner.h/.cpp
|   |   '-- AnalogCommandConditioner.h/.cpp
|   |-- io/
|   |   |-- StdioBridge.h/.cpp
|   |   |-- CommandInterface.h/.cpp
|   |   '-- LcdDisplay.h/.cpp
|   '-- app/
|       '-- ActuatorLabApp.h/.cpp
'-- report/
    |-- main.tex
    '-- plantuml/
```

3.3 Software Module Descriptions

- **AppConfig.h**: compile-time pin assignments, periods, and analog conditioning constants.
- **BinaryActuator**: wraps the digital output pin for the binary actuator state.
- **Stepper28BYJ48Actuator**: generates a non-blocking half-step sequence for a 28BYJ-48 through a ULN2003-style driver path.
- **BinaryCommandConditioner**: implements debounce and stable-state commitment for ON/OFF requests.
- **AnalogCommandConditioner**: implements saturation, low-pass filtering, ramping, and alert flag generation.
- **StdioBridge**: redirects `printf()` output to UART.
- **CommandInterface**: non-blocking command-line reader with support for CR/LF and backspace/delete.
- **LcdDisplay**: formats and writes the two LCD status pages.
- **ActuatorLabApp**: coordinates command parsing, control timing, actuator updates, reporting, and LCD refresh.

3.4 Reference to Source Code

The full implementation is stored in the repository under `lab7/src/`. The report focuses on architecture, behaviour, verification evidence, and system-level explanation rather than embedding large source listings inline.

3.5 Implementation Phase Conclusions

The implementation follows the design directly. One practical issue appeared during validation in VS Code Wokwi: the terminal sent carriage return characters differently than expected. The final **CommandInterface** was therefore extended to accept both CR and LF as line terminators and to handle backspace/delete correctly. This change made terminal-based control reliable while preserving the `printf()` requirement.

4 Testing and Validation

4.1 Build Results

Environment	Flash used	Flash %	RAM used	RAM %
uno	10 768 B	33.4%	1 008 B	49.2%

Table 8: PlatformIO build resource usage (Lab 7)

The firmware builds successfully with `pio run`. Resource usage stays comfortably inside ATmega328P limits.

4.2 Test Execution Report

ID	Scenario	Observed result	Result
T-01	Startup banner	Banner, help, and initial status printed at startup	PASS
T-02	bin on	Binary actuator turns ON after debounce interval	PASS
T-03	bin off	Binary actuator turns OFF after debounce interval	PASS
T-04	bin toggle	Binary state toggles as expected	PASS
T-05	ana 30	Stepper speed ramps smoothly to a low rotation rate	PASS
T-06	ana 85	High-output alert becomes active on LCD/STDIO	PASS
T-07	ana 120	Command saturates to 100% and <code>sat_alert=YES</code> is reported	PASS
T-08	status	One structured status line is printed on demand	PASS
T-09	report on	Periodic reporting resumes every 500 ms	PASS
T-10	report off	Periodic reporting stops without affecting control	PASS
T-11	LCD page rotation	Both LCD pages alternate correctly every 2 seconds	PASS
T-12	Terminal Enter handling	Commands execute correctly with the VS Code Wokwi terminal	PASS
T-13	Terminal backspace handling	Editing typed commands before Enter works correctly	PASS
T-14	Resource limits	Flash and RAM usage remain below Uno limits	PASS
T-15	End-to-end variant C behaviour	Binary and analog actuator control coexist in one project	PASS

Table 9: Test execution results – Lab 7

4.3 Representative Serial Output

Lab 7 - Actuator Control

Commands:

help

status

```

bin on | bin off | bin toggle
ana <0-100>
report on | report off
STAT bin=OFF pending=NO step_req=0% step_sat=0% step_out=0% rpm=0 pos=0 sat_alert=NO
bin on
CMD: binary request -> ON
status
STAT bin=ON pending=NO step_req=0% step_sat=0% step_out=0% rpm=0 pos=0 sat_alert=NO
ana 120
CMD: stepper speed request -> 120%
status
STAT bin=ON pending=NO step_req=120% step_sat=100% step_out=44% rpm=26 pos=184 sat_al
report on
CMD: periodic reporting enabled

```

The trace demonstrates the two distinct command paths clearly: the binary channel commits a debounced stable digital state, while the analog channel exposes requested value, saturated target, current conditioned output, estimated RPM, and step position.

4.4 Performance Analysis

- **Latency:** the 20 ms control period ensures actuator updates remain below the required 100 ms latency.
- **Binary path:** debounce introduces intentional delay, but it remains small enough to satisfy responsiveness constraints.
- **Analog path:** low-pass filtering and ramping improve smoothness at the cost of gradual settling, which is desirable for actuator protection.
- **Terminal robustness:** support for CR, LF, and backspace made the interface usable in VS Code Wokwi without changing the overall architecture.
- **Memory efficiency:** the design remains lightweight because it uses fixed-size state only and no dynamic allocation.

4.5 Testing Phase Conclusions

All defined validation scenarios passed. The project now provides a complete combined actuator-control demonstrator with visible LED feedback, LCD reporting, and structured serial diagnostics based entirely on `printf()`.

General Conclusions

Laboratory Work #7 achieved its objectives. The final project combines binary and analog actuator control in a single Arduino Uno application and therefore matches the most complete interpretation of the assignment variant.

Key outcomes:

1. **Combined actuator variant:** one project controls both a binary actuator and an analog actuator.
2. **STDIO-centred interface:** user interaction and reporting are handled through terminal commands and `printf()` output.
3. **Conditioned command paths:** the binary command path is debounced, while the analog path is saturated, filtered, and ramp-limited.
4. **Readable state exposure:** the LCD and serial terminal expose both control outputs and internal condition flags.
5. **MiKTeX-ready documentation:** the report structure mirrors the previous laboratory while replacing image dependencies with placeholders and PlantUML sources.

Possible improvements:

- Replace the simulated compatible stepper model with the physical 28BYJ-48 5V + ULN2003 hardware stage.
- Add command acknowledgements with timestamps.
- Export runtime traces for plotting actuator response curves.
- Extend the command set with configurable debounce and ramp parameters.

Note on AI Tool Usage

AI-assisted tools were used to support coding productivity and report drafting:

- **GitHub Copilot** – code completion and refactoring assistance.
- **LLM assistance** – technical writing structure, LaTeX drafting, and report polishing.

All generated content was manually reviewed and adapted to the implemented project.

Bibliography

1. Horowitz, P., Hill, W. *The Art of Electronics*, 3rd ed. Cambridge University Press, 2015.
2. Arduino Official Reference. <https://www.arduino.cc/reference/en/>
3. AVR-libc User Manual. <https://www.nongnu.org/avr-libc/user-manual/>
4. PlatformIO Documentation. <https://docs.platformio.org/en/latest/>
5. Wokwi Simulator Documentation. <https://docs.wokwi.com/>
6. LiquidCrystal Library Reference. <https://www.arduino.cc/reference/en/libraries/liquidcrystal/>
7. Lab 7 theory and task notes on actuator control and command-signal conditioning. Technical University of Moldova, 2026.

Annex – Source Code

The full source code is hosted on GitHub:

Repository URL:

<https://github.com/Ekkusuu/embedded-systems-repo/tree/main/lab7>

This repository contains all source files under `lab7/src/`, the PlatformIO configuration (`platformio.ini`), the Wokwi simulation files (`diagram.json`, `wokwi.toml`), and the report sources.

PlantUML diagram source files are stored in:

`lab7/report/plantuml/`

Complete module listing:

- `src/main.cpp` – entry point, object instantiation, `setup()/loop()`
- `src/config/AppConfig.h` – all compile-time constants
- `src/actuators/BinaryActuator.h/.cpp` – binary actuator driver
- `src/actuators/Stepper28BYJ48Actuator.h/.cpp` – 28BYJ-48 stepper motor driver
- `src/signal/BinaryCommandConditioner.h/.cpp` – debounced binary command path

- `src/signal/AnalogCommandConditioner.h/.cpp` – saturation, filtering, and ramping
- `src/io/StdioBridge.h/.cpp` – `fdev_setup_stream` UART bridge
- `src/io/CommandInterface.h/.cpp` – terminal line parser with CR/LF and backspace support
- `src/io/LcdDisplay.h/.cpp` – 16×2 LCD wrapper
- `src/app/ActuatorLabApp.h/.cpp` – application orchestration